



CHAPTER 4

ASP.NET Core Controller and View

GVHD: Ths. Nguyễn Thanh Vũ

CONTENT

- Creating the Project
- Getting Started with Views

CREATING THE PROJECT

Creating the Project

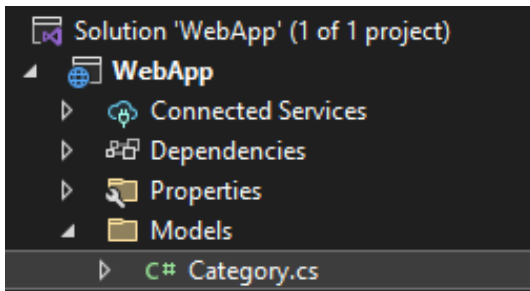
- Open a new PowerShell command prompt and run the commands
 - ❑ `dotnet new globaljson --sdk-version 6.0.100 --output WebApp`
 - ❑ `dotnet new web --no-https --output WebApp --framework net6.0`
 - ❑ `dotnet new sln -o WebApp`
 - ❑ `dotnet sln WebApp add WebApp`

Adding a Data Model

- Adding NuGet Packages to the Project
 - ❑ `dotnet add package Microsoft.EntityFrameworkCore.Design --version 6.0.0`
 - ❑ `dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 6.0.0`

Adding a Data Model

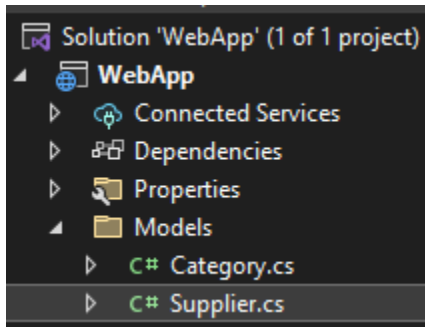
- Create a new folder named Models and add to it a class file named Category.cs



```
public class Category
{
    public long CategoryId { get; set; }
    public string Name { get; set; } = string.Empty;
    public IEnumerable<Product>? Products { get; set; }
}
```

Adding a Data Model

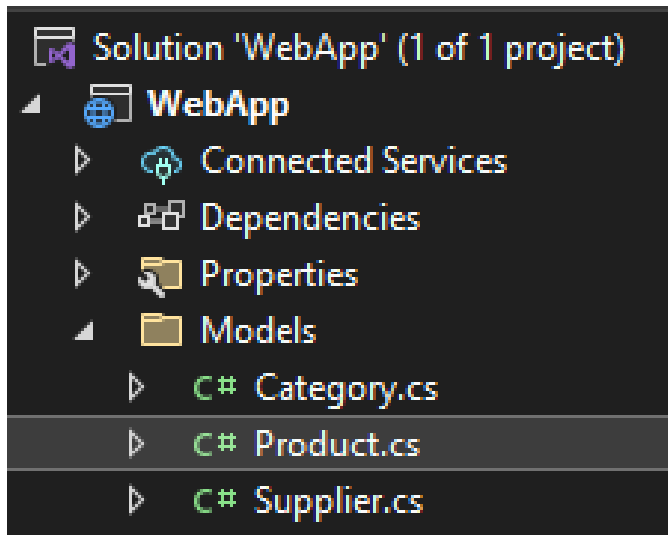
- Add a class called Supplier.cs to the Models folder and use it to define the class shown



```
public class Supplier
{
    public long SupplierId { get; set; }
    public string Name { get; set; } = string.Empty;
    public string City { get; set; } = string.Empty;
    public IEnumerable<Product>? Products { get; set; }
}
```

Adding a Data Model

- Add a class named Product.cs to the Models folder and use it to define the class shown



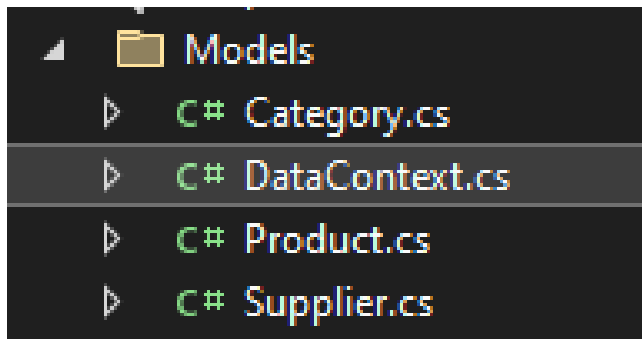
Adding a Data Model

- Add a class named Product.cs to the Models folder and use it to define the class shown

```
public class Product
{
    public long ProductId { get; set; }
    public string Name { get; set; } = string.Empty;
    [Column(TypeName = "decimal(8, 2)")]
    public decimal Price { get; set; }
    public long CategoryId { get; set; }
    public Category? Category { get; set; }
    public long SupplierId { get; set; }
    public Supplier? Supplier { get; set; }
}
```

Entity Framework Core context class

- To create the **Entity Framework Core context class** that will provide access to the database, add a file called DataContext.cs to the Models folder and add the code shown



Entity Framework Core context class

- To create the **Entity Framework Core context class** that will provide access to the database, add a file called DataContext.cs to the Models folder and add the code shown

```
using Microsoft.EntityFrameworkCore;
namespace WebApp.Models
{
    public class DataContext : DbContext
    {
        public DataContext(DbContextOptions<DataContext> opts)
            : base(opts) { }
        public DbSet<Product> Products => Set<Product>();
        public DbSet<Category> Categories => Set<Category>();
        public DbSet<Supplier> Suppliers => Set<Supplier>();
    }
}
```

Preparing the Seed Data

- Add a class called SeedData.cs to the Models folder and add the code shown below to define the seed data that will be used to populate the database.

```
using Microsoft.EntityFrameworkCore;
namespace WebApp.Models
{
    public static class SeedData
    {
        public static void SeedDatabase(DataContext context)
        {
            context.Database.Migrate();
            if (context.Products.Count() == 0 && context.Suppliers.Count() == 0
                && context.Categories.Count() == 0)
            {
                Supplier s1 = new Supplier
                { Name = "Splash Dudes", City = "San Jose" };
                Supplier s2 = new Supplier
                { Name = "Soccer Town", City = "Chicago" };
                Supplier s3 = new Supplier
                { Name = "Chess Co", City = "New York" };
                Category c1 = new Category { Name = "Watersports" };
                Category c2 = new Category { Name = "Soccer" };
                Category c3 = new Category { Name = "Chess" };
            }
        }
    }
}
```

Preparing the Seed Data

```
context.Products.AddRange(  
    new Product  
    {  
        Name = "Kayak",  
        Price = 275,  
        Category = c1,  
        Supplier = s1  
    },  
    new Product  
    {  
        Name = "Lifejacket",  
        Price = 48.95m,  
        Category = c1,  
        Supplier = s1  
    },  
    new Product  
    {  
        Name = "Soccer Ball",  
        Price = 19.50m,  
        Category = c2,  
        Supplier = s2  
    },  
    new Product  
    {  
        Name = "Corner Flags",  
        Price = 34.95m,  
        Category = c2,  
        Supplier = s2  
    },  
    new Product  
    {  
        Name = "Bling-Bling King",  
        Price = 1200,  
        Category = c3,  
        Supplier = s3  
    }  
);  
context.SaveChanges();  
}  
}  
}
```

Configuring Entity Framework Core Services and Middleware

- Make the changes to the **Program.cs** file

```
using Microsoft.EntityFrameworkCore;
using WebApp.Models;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<DataContext>(opts => {
    opts.UseSqlServer(builder.Configuration[
        "ConnectionStrings:ProductConnection"]);
    opts.EnableSensitiveDataLogging(true);
});

var app = builder.Build();
app.MapGet("/", () => "Hello World!");
var context = app.Services.CreateScope().ServiceProvider
    .GetRequiredService<DataContext>();
SeedData.SeedDatabase(context);
app.Run();
```

Configuring Entity Framework Core Services and Middleware

- To define the connection string that will be used for the application's data, add the configuration settings shown below in the appsettings.json file. The connection string should be entered **on a single line**.

```
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft.AspNetCore": "Warning",
6        "Microsoft.EntityFrameworkCore": "Information"
7      }
8    },
9    "AllowedHosts": "*",
10   "ConnectionStrings": {
11     "ProductConnection": "Server=(localdb)\\MSSQLLocalDB;Database=Products;MultipleActiveResultSets=True"
12   }
13 }
```

Creating and Applying the Migration

- To create the migration that will set up the database schema, use a PowerShell command prompt to run the command shown
 - `dotnet tool install --global dotnet-ef --version 6.0.0`
 - `dotnet ef migrations add Initial`
 - `dotnet ef database update`

ADDING THE CSS FRAMEWORK

Adding the CSS Framework

- Run the commands shown below to remove any existing version of the LibMan package and install the version used in this chapter. (You can skip these commands if you installed this version of LibMan in earlier chapters.)
 - `dotnet tool uninstall --global Microsoft.Web.LibraryManager.Cli`
 - `dotnet tool install --global Microsoft.Web.LibraryManager.Cli --version 2.1.175`

Adding the CSS Framework

- To add the Bootstrap CSS framework so that the HTML responses can be styled, run the commands shown below in the WebApp project folder.
 - `libman init -p cdnjs`
 - `libman install bootstrap@5.1.3 -d wwwroot/lib/bootstrap`

GETTING STARTED WITH VIEWS

Configuring the Application

Changing the Configuration in the Program.cs File in the WebApp Folder

```
using Microsoft.EntityFrameworkCore;
using WebApp.Models;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<DataContext>(opts => {
    opts.UseSqlServer(builder.Configuration[
        "ConnectionStrings:ProductConnection"]);
    opts.EnableSensitiveDataLogging(true);
});

builder.Services.AddControllersWithViews();

var app = builder.Build();
app.UseStaticFiles();
app.MapControllers();
app.MapControllerRoute("Default",
    "{controller=Home}/{action=Index}/{id?}");

var context = app.Services.CreateScope().ServiceProvider
    .GetRequiredService<DataContext>();
SeedData.SeedDatabase(context);

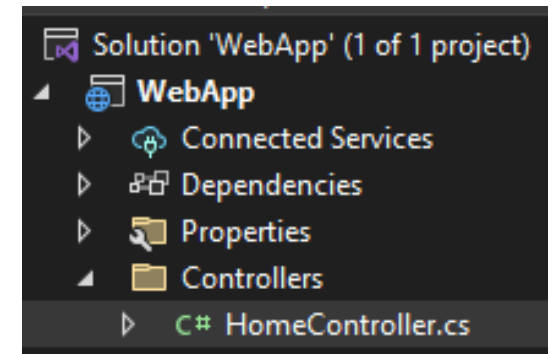
app.Run();
```

Creating an HTML Controller

To create an HTML controller, add a class file named HomeController.cs to the Controllers folder with the statements shown

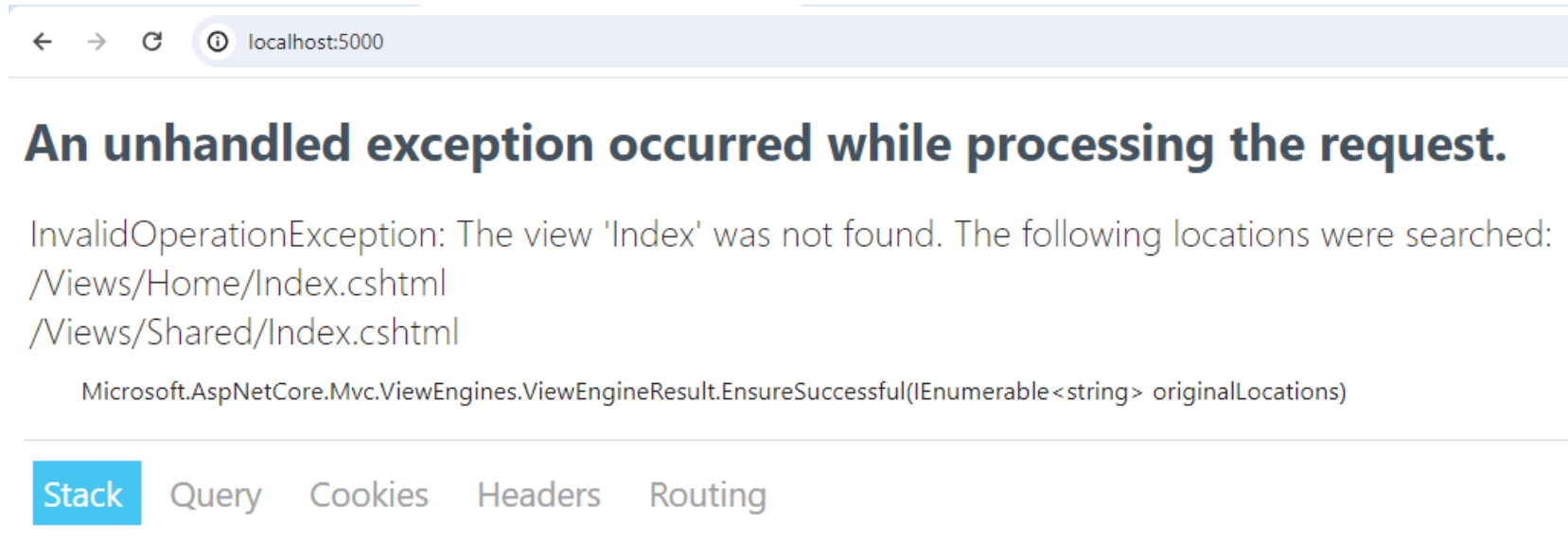
```
using Microsoft.AspNetCore.Mvc;
using WebApp.Models;

namespace WebApp.Controllers
{
    public class HomeController : Controller
    {
        private DataContext context;
        public HomeController(DataContext ctx)
        {
            context = ctx;
        }
        public async Task<IActionResult> Index(long id = 1)
        {
            return View(await context.Products.FindAsync(id));
        }
    }
}
```



Creating an HTML Controller

When run the program, we will see this error:

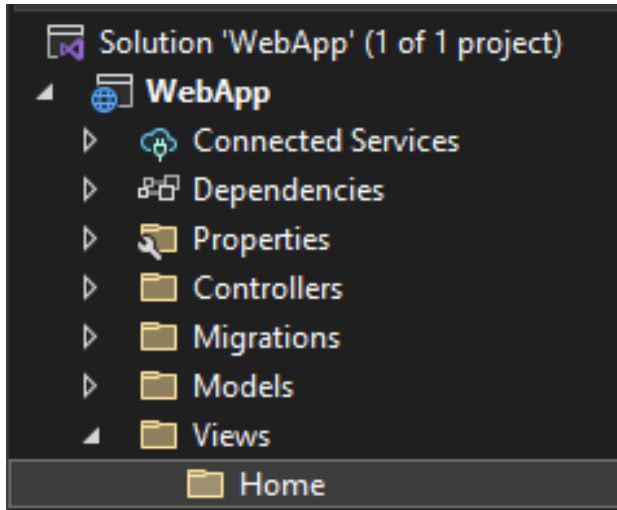


No View in Home or
Shared folder yet

CREATING A RAZOR VIEW

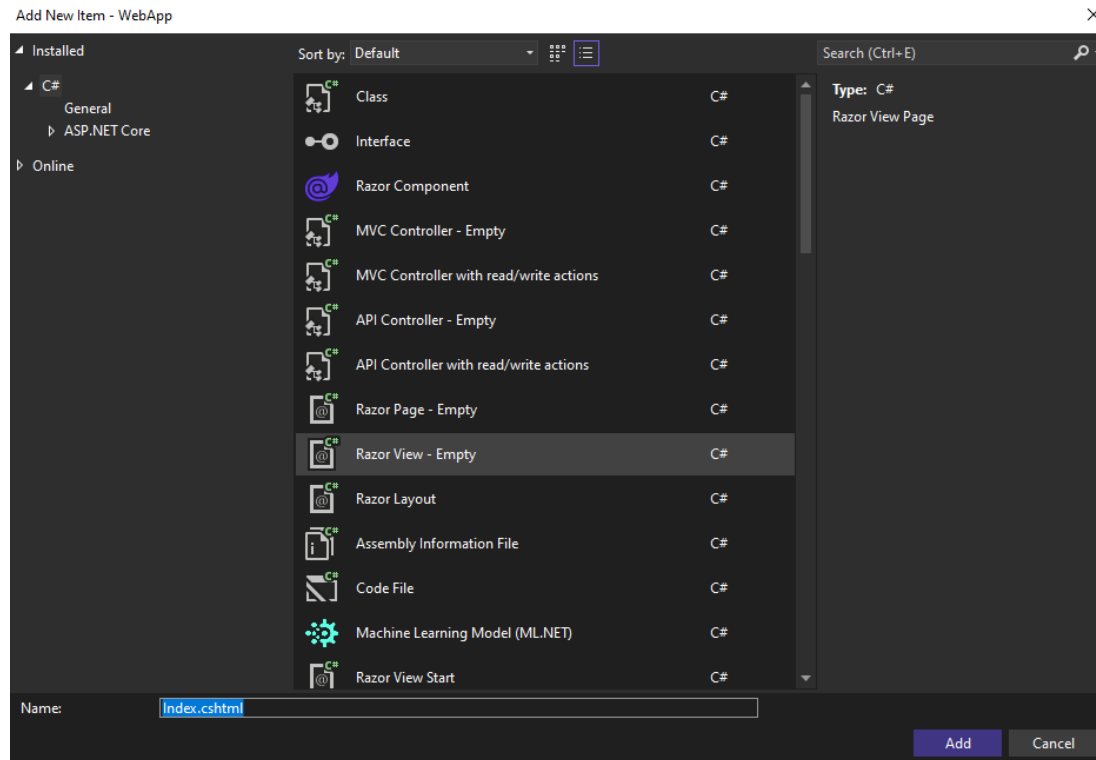
Creating a Razor View

To provide the MVC Framework with a view to display,
create the Views/Home folder



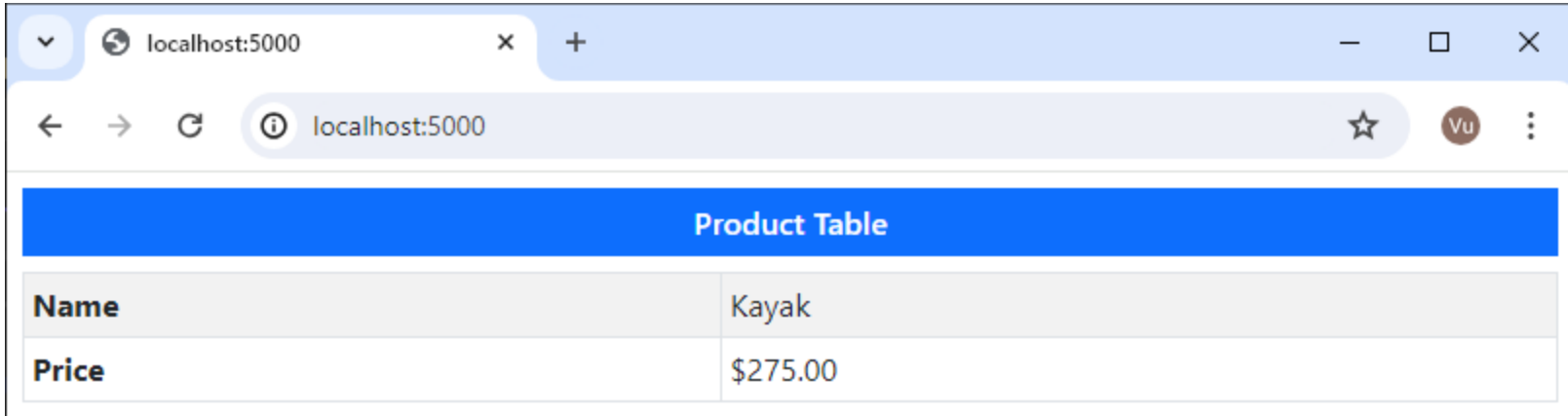
Creating a Razor View

If you are using Visual Studio, create the view by right-clicking the Views/Home folder, selecting Add ➤ New Item from the pop-up menu, and selecting the Razor View - Empty item in the ASP.NET Core ➤ Web category



Creating a Razor View

Run the program and see result:

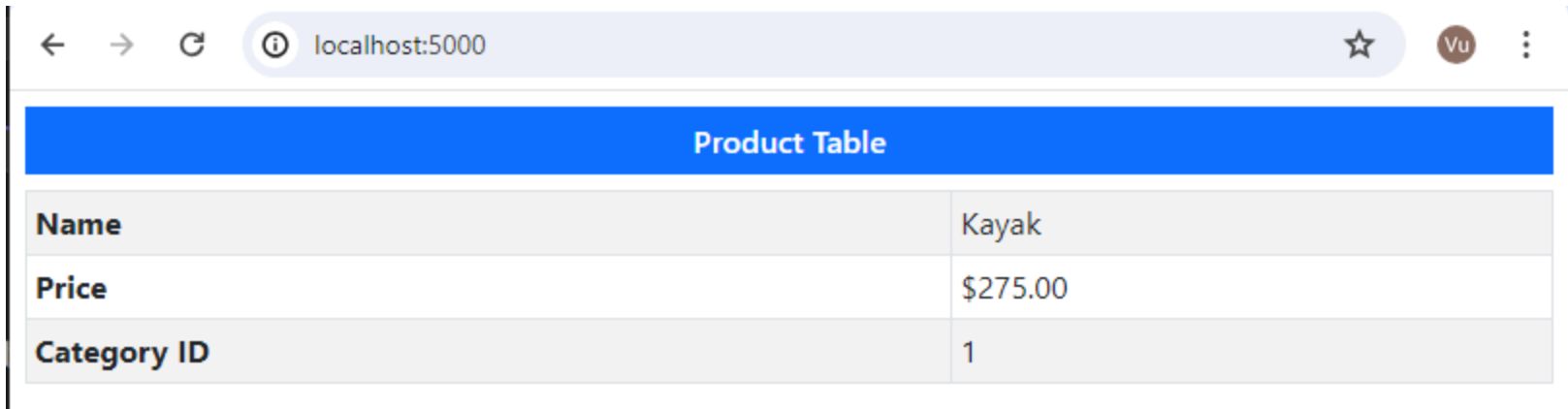


A screenshot of a web browser window. The address bar shows 'localhost:5000'. The page content features a table with a blue header and two data rows. The header row is labeled 'Product Table'. The first data row has 'Name' and 'Kayak'. The second data row has 'Price' and '\$275.00'.

Product Table	
Name	Kayak
Price	\$275.00

Modifying a Razor View

The `dotnet watch` command detects and recompiles Razor Views automatically, meaning that the ASP.NET Core runtime **doesn't have to be restarted** when views are edited



A screenshot of a web browser window. The address bar shows 'localhost:5000'. The page displays a table with a blue header 'Product Table'. The table has three rows: 'Name' with value 'Kayak', 'Price' with value '\$275.00', and 'Category ID' with value '1'.

Product Table	
Name	Kayak
Price	\$275.00
Category ID	1

Using Shared Views

- ❑ When the Razor view engine locates a view, it looks in the **View/[controller] folder** and then the **Views/Shared folder**. This search pattern means that views that contain common content can be shared between controllers, avoiding duplication.
- ❑ To see how this process works, add a Razor View file named **Common.cshtml** to the **Views/Shared folder** with the content shown below

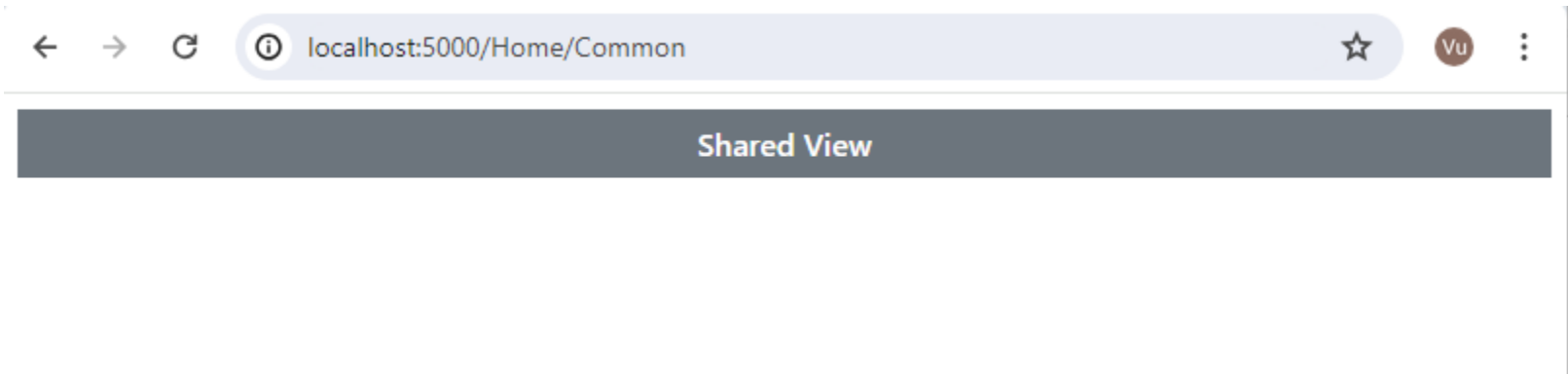
Using Shared Views

```
<!DOCTYPE html>
<html>
<head>
  <link href="/lib/bootstrap/css/bootstrap.min.css" rel="stylesheet" />
</head>
<body>
  <h6 class="bg-secondary text-white text-center m-2 p-2">Shared View</h6>
</body>
</html>
```


Using Shared Views

- ❑ Next, add an action method to the **Home controller** that uses the new view

```
public IActionResult Common()
{
    return View();
}
```



Working with Razor Views

- ❑ Razor Views contain HTML elements and C# expressions. Expressions are mixed in with the HTML elements and denoted with the **@ character**, like this:

```
...  
<tr><th>Name</th><td>@Model?.Name</td></tr>  
...
```

Working with Razor Views

- ❑ When the view is used to generate a response, the expressions are evaluated, and the results are included in the content sent to the client. This expression gets the name of the Product view model object provided by the action method and produces output like this

```
...  
<tr><th>Name</th><td>Corner Flags</td></tr>  
...
```

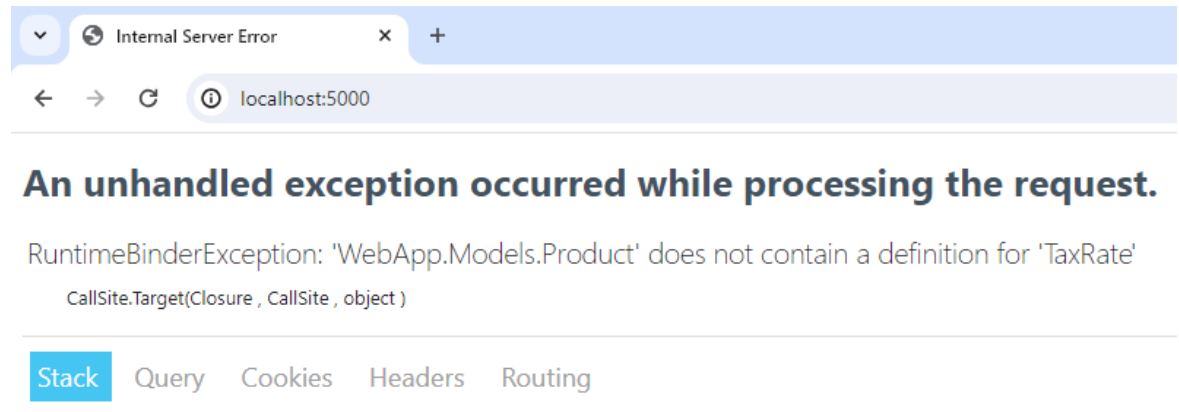
SETTING THE VIEW MODEL TYPE

Setting the View Model Type

- ❑ The generated class for the Index.cshtml file is derived from `RazorPage<T>`, but **Razor doesn't know what type will be used** by the action method for the view model, so it has selected `dynamic` as the generic type argument. This means that the **@Model expression can be used with any property or method name**, which is evaluated at runtime when a response is generated. To demonstrate what happens when a nonexistent member is used, add the content shown

Setting the View Model Type

```
<!DOCTYPE html>
<html>
<head>
  <link href="/lib/bootstrap/css/bootstrap.min.css" rel="stylesheet" />
</head>
<body>
  <h6 class="bg-primary text-white text-center m-2 p-2">Product Table</h6>
  <div class="m-2">
    <table class="table table-sm table-striped table-bordered">
      <tbody>
        <tr><th>Name</th><td>@Model?.Name</td></tr>
        <tr><th>Price</th><td>@Model?.Price.ToString("c")</td></tr>
        <tr><th>Category ID</th><td>@Model?.CategoryId</td></tr>
        <tr><th>Tax Rate</th><td>@Model?.TaxRate</td></tr>
      </tbody>
    </table>
  </div>
</body>
</html>
```



UNDERSTANDING THE RAZOR SYNTAX

Understanding the Razor Syntax

- ❑ The Razor compiler **separates** the static fragments of HTML from the C# expressions, which are then handled separately in the generated class file. There are several types of expression that can be included in views, which I describe in the sections that follow

Understanding Directives

- ❑ Directives are expressions that give instructions to the Razor view engine. **The @model expression is a directive**, for example, that tells the view engine to use a specific type for the view model, while the **@using directive** tells the view engine to import a namespace

Understanding Directives

Table 21-5. *Useful Razor Directives*

Name	Description
@model	This directive specifies the type of the view model.
@using	This directive imports a namespace.
@page	This directive denotes a Razor Page, described in Chapter 23.
@section	This directive denotes a layout section, as described in Chapter 22.
@addTagHelper	This directive adds tag helpers to a view, as described in Chapter 25.
@namespace	This directive sets the namespace for the C# class generated from a view.
@functions	This directive adds C# properties and methods to the C# class generated from a view and is commonly used in Razor Pages, as described in Chapter 23.
@attribute	This directive adds an attribute to the C# class generated from a view. I use this feature to apply authorization restrictions in Chapter 38.
@implements	This directive declares that the C# class generated from a view implements an interface. This feature is demonstrated in Chapter 36.
@inherits	This directive sets the base class for the C# class generated from a view. This feature is demonstrated in Chapter 36.
@inject	This directive provides a view with direct access to a service through dependency injection. This feature is demonstrated in Chapter 23.

Understanding Content Expressions

- ❑ Razor content expressions produce content that is included in the output generated by a view. Table 21-6 describes the most useful content expressions, which are demonstrated in the sections that follow

Understanding Content Expressions

Table 21-6. *Useful Razor Content Expressions*

Name	Description
@<expression>	This is the basic Razor expression, which is evaluated, and the result it produces is inserted into the response.
@if	This expression is used to select regions of content based on the result of an expression. See the “Using Conditional Expressions” section for examples.
@switch	This expression is used to select regions of content based on the result of an expression. See the “Using Conditional Expressions” section for examples.
@foreach	This expression generates the same region of content for each element in a sequence. See the “Enumerating Sequences” for examples.
@{ ... }	This expression defines a code block. See the “Using Razor Code Blocks” section for an example.
@:	This expression denotes a section of content that is not enclosed in HTML elements. See the “Using Conditional Expressions” section for an example.
@try	This expression is used to catch exceptions.
@await	This expression is used to perform an asynchronous operation, the result of which is inserted into the response. See Chapter 24 for examples.

THE NULL CONDITIONAL OPERATOR

- When the @Model expression is used, the null conditional operator (the ? operator) is required to guard against null model values, like this:

```
...  
<tr><th>Name</th><td>@Model?.Name</td></tr>  
...
```

Using Conditional Expressions

- ❑ Razor supports conditional expressions, which means that the output can be tailored based on the view model. This technique is at the heart of Razor and allows you to create complex and fluid responses from views that are simple to read and maintain.

```
@if (Model?.Price > 200) {  
    <tr><th>Name</th><td>Luxury @Model?.Name</td></tr>  
} else {  
    <tr><th>Name</th><td>Basic @Model?.Name</td></tr>  
}
```

Using Conditional Expressions

- ❑ Notice that the `@` prefix isn't required to access a `Model` property in the condition.

```
...  
@if (Model?.Price > 200) {  
...  
}
```

- ❑ But the `@` prefix is required inside the if and else clauses, like this

```
...  
<tr><th>Name</th><td>Luxury @Model?.Name</td></tr>  
...
```


Using Conditional Expressions

- ❑ Razor also supports **@switch expressions**, which can be a more concise way of handling multiple conditions

```
@switch (Model?.Name) {  
    case "Kayak":  
        <tr><th>Name</th><td>Small Boat</td></tr>  
        break;  
    case "Lifejacket":  
        <tr><th>Name</th><td>Flotation Aid</td></tr>  
        break;  
    default:  
        <tr><th>Name</th><td>@Model?.Name</td></tr>  
        break;  
}
```

the @: prefix

- ❑ The Razor compiler needs help with literal values that are not enclosed in HTML elements, requiring the @: prefix, like this

```
...  
@:Small Boat  
...
```

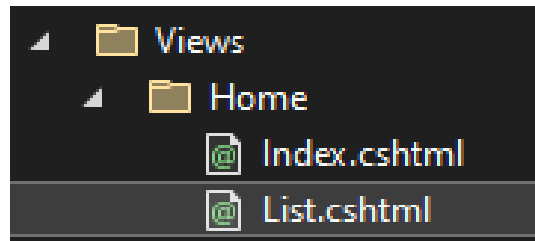
Enumerating Sequences

- ❑ The Razor **@foreach expression** generates content for each object in an array or a collection, which is a common requirement when processing data. Let's add an action method to the Home controller that produces a sequence of objects

```
0 references  
public IActionResult List()  
{  
    return View(context.Products);  
}
```

Enumerating Sequences

- ❑ The new action is called List, and it provides its view with the sequence of Product objects obtained from the Entity Framework Core data context. Add a Razor View file named List.cshtml to the Views/Home folder and add the content shown below



Using Razor Code Blocks

- ❑ Code blocks are regions of C# content that do not generate content but that can be useful to perform tasks that support the expressions that do

Using Razor Code Blocks

```
@using WebApp.Models
@model IEnumerable<Product>
@{
    decimal average = Model?.Average(p => p.Price) ?? 0;
}
<!DOCTYPE html>
<html>
<head>
    <link href="/lib/bootstrap/css/bootstrap.min.css" rel="stylesheet" />
</head>
<body>
    <h6 class="bg-secondary text-white text-center m-2 p-2">Products</h6>
    <div class="m-2">
        <table class="table table-sm table-striped table-bordered">
            <thead>
                <tr><th>Name</th><th>Price</th></tr>
            </thead>
            <tbody>
                @foreach (Product p in Model ?? Enumerable.Empty<Product>())
                {
                    <tr>
                        <td>@p.Name</td>
                        <td>@p.Price</td>
                        <td>
                            @((p.Price / average * 100).ToString("F1"))
                            % of average
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
</body>
</html>
```